

Documento de Especificação de Requisitos

[HOSPED – Gestão de Quartos e Hospedagem] [1.0]

SUMÁRIO

INTRODUÇÃO	3
VISÃO GERAL DO SISTEMA	3
ATORES DO SISTEMA	3
REQUISITOS FUNCIONAIS – NÍVEL USUÁRIO	4
REGRAS DE NEGÓCIO	4
DETALHAMENTO DOS CASOS DE USO	6
MATRIZ DE RASTREABILIDADE	22
CASOS DE USO	22
DIAGRAMA ENTIDADE-RELACIONAMENTO	23

INTRODUÇÃO

O HOSPED é um sistema de gestão de quartos, hóspedes e reservas desenvolvido exclusivamente para hotéis. O sistema conta com as funcionalidades de cadastro de quartos, hospedes, reservas de quartos e limpeza mais reposição de itens nos quartos. Futuramente é possível desenvolver um sistema de gestão de estoques mais robusto e completo, permitindo que os funcionários identifiquem itens que irão faltar em breve.

VISÃO GERAL DO SISTEMA

O sistema HOSPED é exclusivo para uso interno do hotel, sendo acessado pelos funcionários no seu dia a dia para realizar reservas e controlar os quartos, sendo possível visualizar quem está nos quartos, saídas dos hospedes, garantindo que aquele quarto pode passar pela limpeza e controlar a troca dos itens dos quartos.

O sistema contará com as seguintes funcionalidades:

- 1- Cadastro de funcionário do hotel com o nome, CPF e senha de acesso.
- 2- Cadastro de quartos que conterà campos padrão como numeração, itens de higiene, quantidade de camas de solteiro e quantidade de camas de casal, adição de itens e quantidade do item que deve ser deixada no quarto. Além disso, quando o quarto estiver reservado ele entra na lista de reservados no momento e possui um booleano que indica se o hóspede está no quarto ou não garantindo a limpeza dos quartos.
- 3- Cadastro de hóspede que conterà os campos nome, CPF, telefone, e-mail, endereço (divido nos campos necessários). Esse hóspede ficará registrado na base do sistema e em caso de realização de reserva possuíra um contador que indica quantas vezes ele já se hospedou.
- 4- Cadastro de reservas que conterà os dados do quarto, data da reserva, data fim da reserva, dados do hóspede. A reserva gerará as informações necessárias para o pagamento e é possível encaminhar para o hóspede via e-mail os dados do pagamento para que ele possa realizar o pagamento da reserva dele.

ATORES DO SISTEMA

Ator	Descrição	Funções
Administrador	Usuário com privilégio	Mantém o cadastro de todos os funcionários e já vêm cadastrado no banco de dados e possui também as funções dos funcionários
Funcionário	Usuário cadastrado pelo administrador	Realiza o cadastro de quartos hóspedes e reservas. Dá check-in e check-out. Adiciona observações e finaliza as limpezas
Hóspede	Pessoa que irá se hospedar no hotel	Usuário que realiza o pagamento
Sistema	Processa informações	Processa dados e pagamentos

REQUISITOS FUNCIONAIS – NÍVEL USUÁRIO

ID	Descrição	Versão	Desenvolvedor	Status
RF001	O sistema deve permitir que o Administrador mantenha o cadastro de funcionários.	1.0	Nicoli	Ativo
RF002	O sistema deve permitir manter cadastro de quartos	1.0	Nicoli	Ativo
RF003	O sistema deve permitir registrar objetos e suas quantidades em cada quarto	1.0	Nicoli	Ativo
RF004	O sistema deve permitir registrar reservas vinculadas a um hóspede e a um quarto	1.0	Nicoli	Ativo
RF005	O sistema deve exibir o histórico de reservas de cada hóspede	1.0	Samuel	Ativo
RF006	O sistema deve permitir registrar hóspedes	1.0	Samuel	Ativo
RF007	O sistema deve permitir realizar check-in e check-out de reservas	1.0	Samuel	Ativo
RF008	O sistema deve exibir os quartos aguardando limpeza após check-out	1.0	Nicoli	Ativo
RF009	O sistema deve permitir registrar comentários e arquivos ao encerrar uma reserva	1.0	Samuel	Ativo
RF010	O sistema deve permitir ao hóspede escolher, no ato da reserva, entre pagar antecipadamente (via Pix ou cartão) ou pagar presencialmente no balcão no momento do check-in	1.0	Samuel	Ativo
RF011	Caso o hóspede opte pelo pagamento antecipado, o sistema deve processar o pagamento via Pix, cartão de crédito ou débito	1.0	Lucas	Ativo
RF012	Caso o hóspede opte pelo pagamento antecipado, o sistema deve enviar as informações de pagamento ao e-mail cadastrado	1.0	Lucas	Ativo

REQUISITOS NÃO FUNCIONAIS – NÍVEL USUÁRIO

ID	Descrição	Versão	Desenvolvedor	Status
RNF001	O sistema deve responder às requisições HTTP em no máximo 3 segundos sob condições normais de uso.	1.0	Nicoli	Ativo
RNF002	O sistema deve estar disponível 99% do tempo durante o horário de operação do hotel.	1.0	Nicoli	Ativo
RNF003	Todas as requisições aos microserviços devem ser autenticadas via token JWT validado pelo api-gateway. Senhas de funcionários devem ser armazenadas com hash BCrypt.	1.0	Nicoli	Ativo
RNF004	A arquitetura de microserviços deve permitir que cada serviço seja escalado de forma independente conforme a demanda.	1.0	Nicoli	Ativo
RNF005	O código deve seguir os princípios de separação de responsabilidades, com camadas distintas (controller, service, repository), facilitando manutenção e evolução independente de cada microserviço.	1.0	Samuel	Ativo
RNF006	Cada microserviço deve possuir cobertura mínima de 50% por testes automatizados	1.0	Samuel	Ativo
RNF007	O sistema deve ser executado via Docker, garantindo consistência de ambiente entre desenvolvimento e produção.	1.0	Samuel	Ativo
RNF008	Informações pessoais dos hóspedes (nome, e-mail) devem trafegar apenas internamente entre os microserviços.	1.0	Nicoli	Ativo

REGRAS DE NEGÓCIO

ID	Descrição	Versão	Desenvolvedor	Status
RN001	Um funcionário deve ter nome, cargo (Administrador ou funcionário) e credenciais de acesso ao sistema com senha e cpf	1.0	Nicoli	Ativo
RN002	Um quarto deve ter número, tipo (simples, duplo, suíte) e capacidade máxima de hóspedes.	1.0	Nicoli	Ativo
RN003	Não é permitido excluir um quarto que possua reservas ativas	1.0	Nicoli	Ativo
RN004	Ao registrar um objeto em um quarto, a quantidade deve ser obrigatoriamente maior que zero	1.0	Nicoli	Ativo
RN005	Após o check-out, o quarto deve automaticamente receber o status "aguardando limpeza"	1.0	Nicoli	Ativo
RN006	A reserva deve registrar obrigatoriamente data de check-in, check-out e hóspede responsável	1.0	Samuel	Ativo
RN007	A data de check-out deve ser posterior à data de check-in	1.0	Samuel	Ativo
RN008	Não é permitido reservar um quarto já ocupado ou em limpeza no período solicitado.	1.0	Samuel	Ativo

RN009	O status da reserva deve seguir o fluxo: Pendente → Ativa → Encerrada (pagamento no balcão) ou Pendente → Pago → Ativa → Encerrada (pagamento antecipado)	1.0	Samuel	Ativo
RN010	A confirmação do pagamento antecipado é obrigatória para que a reserva avance de "Pendente" para "Pago" — reservas com pagamento no balcão avançam direto para "Ativa" no check-in	1.0	Ativo	Ativo
RN011	O cancelamento automático por falta de confirmação de pagamento se aplica somente às reservas com pagamento antecipado	1.0	Ativo	Ativo
RN012	Um hóspede deve ter nome completo, CPF, e-mail e telefone obrigatórios no cadastro. Além disso podem ser informadas alergias e condições médicas. ALER	1.0	Samuel	Ativo
RN013	Não é permitido excluir um hóspede com reservas ativas ou histórico de estadias	1.0	Samuel	Ativo
RN014	Não é permitido dois quartos com o mesmo número.	1.0	Nicoli	Ativo
RN015	A senha do funcionário deve ter no mínimo 8 caracteres, letra maiúscula, letra minúscula, número e caractere especial	1.0	Nicoli	Ativo
RN16	O usuário deve conseguir somente alterar a sua senha	1.0	Nicoli	Ativo

ID	Descrição	Versão	Data	Fonte	Desenvolvedor	Status
RNF001		1.0				Ativo
RNF002		1.0				Ativo
RNF003		1.0				Ativo

DETALHAMENTO DOS CASOS DE USO

CDU001	Login
DESCRIÇÃO: Usuário loga na sua conta para utilizar o sistema	
ATOES: Administrador, Funcionário	
PRÉ-CONDIÇÕES: Usuário cadastrado no sistema	
FLUXO BÁSICO: 1- Usuário informa CPF e senha 2- O sistema valida os dados informados 3- O sistema retorna o token JWT	
FLUXO DE EXCEÇÕES: Cpf ou Senha inválidos	
ESTRUTURA DE DADOS	

CPF e Senha
PÓS CONDIÇÕES Usuário logado no sistema e com acesso às funcionalidades do seu perfil

CDU002	Cadastrar Funcionário
DESCRIÇÃO: Cadastrar novos funcionários no sistema	
ATORES: Administrador	
PRÉ-CONDIÇÕES: Estar autenticado como Administrador	
FLUXO BÁSICO: 1- O Administrador informar nome, CPF, cargo e senha 2- O Sistema valida as informações 3- O Sistema cadastra o funcionário no banco de dados	
FLUXO DE EXCEÇÕES: CPF duplicado, senha fraca ou funcionário sem permissão → retorna erro	
ESTRUTURA DE DADOS Nome, CPF, cargo e senha	
PÓS CONDIÇÕES Usuário cadastrado no sistema	

CDU003	Cadastrar Quarto
DESCRIÇÃO: Cadastro de novos quartos do hotel	
ATORES: Administrador, Funcionário	
PRÉ-CONDIÇÕES: Estar autenticado	
FLUXO BÁSICO: 1- O usuário informa número, tipo e capacidade do quarto. 2- O Sistema valida as informações 3- O Sistema cadastra o quarto no banco de dados	
FLUXO DE EXCEÇÕES: Número do quarto duplicado	
ESTRUTURA DE DADOS Número, tipo e capacidade.	
PÓS CONDIÇÕES Quarto cadastrado no sistema	

CDU004	Gerenciar Itens do Quarto
DESCRIÇÃO: Adicionar itens ao quarto.	
ATOES: Administrador, Funcionário	
PRÉ-CONDIÇÕES: Quarto cadastrado	
FLUXO BÁSICO: 1- O usuário informa nome e quantidade do item 2- O sistema valida as informações 3- O sistema adiciona o item ao quarto	
FLUXO DE EXCEÇÕES: Quantidade 0 ou negativa do item.	
ESTRUTURA DE DADOS Nome do item, quantidade	
PÓS CONDIÇÕES Item adicionado ao quarto	

CDU005	Realizar Checkout
DESCRIÇÃO: Incluir liberação do quarto no sistema e alterar a situação do quarto	
ATOES: Administrador, Funcionário	
PRÉ-CONDIÇÕES: Quarto ocupado	
FLUXO BÁSICO: 1- Seleciona o quarto 2- O sistema altera a situação do quarto para aguardando limpeza	
FLUXO DE EXCEÇÕES: Quarto não localizado	
ESTRUTURA DE DADOS Quarto reservado	
PÓS CONDIÇÕES Quarto aguardando limpeza	

CDU006	Finalizar Limpeza
DESCRIÇÃO: Quarto limpo	
ATOES: Administrador, Funcionário	
PRÉ-CONDIÇÕES: Quarto com status AGUARDANDO LIMPEZA	
FLUXO BÁSICO:	

1- Quarto selecionado 2- Sistema muda status para DISPONÍVEL
FLUXO DE EXCEÇÕES: Quarto não encontrado
ESTRUTURA DE DADOS Quarto aguardando limpeza
PÓS CONDIÇÕES: Quarto limpo e livre para reservas

CDU007	Listar Quartos Aguardando Limpeza
DESCRIÇÃO: Lista dos quartos que receberam check-out e não estão limpos.	
ATORES: Administrador, Funcionário	
PRÉ-CONDIÇÕES: Estar autenticado	
FLUXO BÁSICO: 1- O sistema retorna a lista de quartos com status AGUARDANDO LIMPEZA	
FLUXO DE EXCEÇÕES:	
ESTRUTURA DE DADOS: Lista de quartos aguardando limpeza	
PÓS CONDIÇÕES: Lista impressa	

CDU008	Alterar Senha
DESCRIÇÃO: Alterar a senha do usuário funcionário	
ATORES: Administrador, Funcionário	
PRÉ-CONDIÇÕES: Estar autenticado como administrador	
FLUXO BÁSICO: 1- O Administrador loga no sistema 2- O administrador seleciona o usuário a ser alterado 3- O administrador informa a senha antiga e a nova senha 4- O sistema valida a senha 5- A senha é atualizada	
FLUXO DE EXCEÇÕES: Senha atual incorreta ou nova senha fraca	
ESTRUTURA DE DADOS	

Senha antiga e senha nova.
PÓS CONDIÇÕES Nova senha cadastrada

CDU009	Excluir Funcionário
DESCRIÇÃO: Exclusão de funcionário	
ATOES: Administrador	
PRÉ-CONDIÇÕES: Estar autenticado como Administrador	
FLUXO BÁSICO: 1- Selecionar o funcionário 2- O Sistema remove o funcionário	
FLUXO DE EXCEÇÕES: Funcionário não encontrado	
ESTRUTURA DE DADOS Dados do funcionário	
PÓS CONDIÇÕES Funcionário removido do sistema sem acesso para logar.	

CDU0010	Excluir Quarto
DESCRIÇÃO: Exclusão de quarto	
ATOES: Administrador, Funcionário	
PRÉ-CONDIÇÕES: Quarto não ocupado	
FLUXO BÁSICO: 1- Selecionar um quarto 2- O Sistema remove o quarto	
FLUXO DE EXCEÇÕES: Quarto ocupado	
ESTRUTURA DE DADOS Quarto	
PÓS CONDIÇÕES Quarto removido do sistema	

CDU0011	Processar Reserva
----------------	-------------------

DESCRIÇÃO: O sistema recebe um evento de reserva criada via RabbitMQ e cria um pagamento pendente para o hóspede
ATORES: ms-reservas(sistema)
PRÉ-CONDIÇÕES: Reserva criada no ms-reservas e o evento reserva.criada publicado na fila.
FLUXO BÁSICO: 1. ms-reservas publica evento na fila ms-pagamentos.reservas-criadas. 2. ms-pagamentos consome o evento com os dados da reserva. 3. Sistema calcula o valor total (valorDiaria × número de dias). 4. Sistema cria um pagamento com status PENDENTE e data de expiração (8h). 5. Sistema salva o pagamento no banco de dados. 6. Sistema envia e-mail ao hóspede com link de confirmação.
FLUXO DE EXCEÇÕES: FE01 - Falha no envio de e-mail: sistema lança RuntimeException e o pagamento não é criado.
ESTRUTURA DE DADOS: entrada:reservaId, nomeHospede, emailHospede, valorDiaria, checkIn, checkOut, metodoPagamento. Saída: Pagamento salvo com id (UUID), status PENDENTE, dataExpiracao.
PÓS CONDIÇÕES: Pagamento criado com status PENDENTE. E-mail enviado ao hóspede.

CDU0012	Confirmar Pagamento
DESCRIÇÃO: O hóspede clica no link recebido por e-mail para confirmar o pagamento da reserva.	
ATORES: Hóspede	
PRÉ-CONDIÇÕES: Pagamento existe com status PENDENTE e dentro do prazo de expiração.	
FLUXO BÁSICO: 1. Hóspede confirma o pagamento via email. 2. Sistema busca o pagamento pelo ID. 3. Sistema valida que o status é PENDENTE. 4. Sistema valida que o pagamento não expirou. 5. Sistema atualiza status para APROVADO e registra dataAtualizacao. 6. Sistema publica evento pagamento.aprovado no RabbitMQ. 7. Sistema envia e-mail de confirmação ao hóspede.	

8. Sistema retorna mensagem de sucesso.
FLUXO DE EXCEÇÕES: FE01 - Pagamento não encontrado: retorna 404 Not Found. FE02 - Pagamento já aprovado ou expirado: retorna 400 Bad Request. FE03 - Pagamento expirado pelo prazo: retorna 400 Bad Request.
ESTRUTURA DE DADOS: Entrada: id Saída: String com mensagem de confirmação (200 OK).
PÓS CONDIÇÕES: Pagamento com status APROVADO. Evento publicado no RabbitMQ. E-mail de confirmação enviado.

CDU0013	Atualizar Status do Pagamento
DESCRIÇÃO: Administrador atualiza manualmente o status de um pagamento.	
ATORES: administrador, funcionário	
PRÉ-CONDIÇÕES: Pagamento existe no sistema.	
FLUXO BÁSICO: 1. Administrador envia PATCH /pagamentos/{id}/status?status={novoStatus}. 2. Sistema busca o pagamento pelo ID. 3. Sistema atualiza o status e registra dataAtualizacao. 4. Sistema publica evento pagamento.processado no RabbitMQ. 5. Se APROVADO: sistema envia e-mail de pagamento aprovado. 6. Se EXPIRADO: sistema envia e-mail de reserva expirada. 7. Sistema retorna 204 No Content.	
FLUXO DE EXCEÇÕES: FE01 - Pagamento não encontrado: retorna 404 Not Found. FE02 - Status inválido: retorna 400 Bad Request.	
ESTRUTURA DE DADOS: Entrada: id (path variable), status (query param: PENDENTE APROVADO EXPIRADO). Saída: 204 No Content.	
PÓS CONDIÇÕES: Status do pagamento atualizado. Evento publicado. E-mail enviado conforme o novo status.	

CDU0014	Buscar Pagamento por ID
----------------	-------------------------

DESCRIÇÃO: Usuário busca os detalhes de um pagamento específico pelo seu identificador.
ATORES: administrador, funcionário
PRÉ-CONDIÇÕES: Usuário possui o ID do pagamento.
FLUXO BÁSICO: 1. Usuário envia GET /pagamentos/{id}. 2. Sistema busca o pagamento pelo ID. 3. Sistema retorna os dados do pagamento em formato DTO.
FLUXO DE EXCEÇÕES: FE01 - Pagamento não encontrado: retorna 404 Not Found.
ESTRUTURA DE DADOS: Entrada: id. Saída (PagamentoResponseDTO): id, reservaId, valor, status, metodoPagamento, dataCriacao, dataAtualizacao, dataExpiracao, nomeHospede.
PÓS CONDIÇÕES: Dados do pagamento retornados ao usuário.

CDU0015	Buscar Pagamentos por Reserva
DESCRIÇÃO: Usuário busca todos os pagamentos vinculados a uma reserva específica.	
ATORES: administrador, funcionário	
PRÉ-CONDIÇÕES: Usuário possui o ID da reserva.	
FLUXO BÁSICO: 1. Usuário envia GET /pagamentos/reserva/{reservaId}. 2. Sistema busca todos os pagamentos com o reservaId informado. 3. Sistema retorna a lista de pagamentos em formato DTO.	
FLUXO DE EXCEÇÕES: FE01 - Nenhum pagamento encontrado: retorna lista vazia.	
ESTRUTURA DE DADOS: Entrada: reservaId (path variable). Saída: Lista de PagamentoResponseDTO.	
PÓS CONDIÇÕES: Lista de pagamentos retornada ao usuário.	

CDU0016	Criar reserva
DESCRIÇÃO: Usuário cria uma reserva vinculada a um hóspede e a um quarto, com modalidade de pagamento no hotel ou antecipado.	
ATORES: administrador, funcionário	

PRÉ-CONDIÇÕES: Hóspede deve existir. Deve ser informado quartoId ou quartoServicoId. As datas devem ser válidas.	
FLUXO BÁSICO: 1. Usuário informa os dados da reserva. 2. Usuário envia POST /api/v1/reservas. 3. Sistema valida hóspede, datas, quarto, modalidade de pagamento e, quando houver quartoServicoId, consulta disponibilidade externa. 4. Sistema salva a reserva com status PENDENTE, registra histórico e publica evento de reserva criada. 5. Se o pagamento for antecipado, o sistema também publica evento para criação do pagamento. 6. O sistema retorna a reserva criada.	
FLUXO DE EXCEÇÕES: FE01 - Hóspede não encontrado: retorna erro de recurso não encontrado. FE02 - quartoId e quartoServicoId ausentes: retorna erro de regra de negócio. FE03 - checkOutData não posterior a checkInData: retorna erro de regra de negócio. FE04 - Quarto indisponível na integração externa: retorna erro de integração. FE05 - Pagamento antecipado sem valorDiaria ou metodoPagamento: retorna erro de regra de negócio. FE06 - Dados obrigatórios inválidos: retorna erro de validação.	
ESTRUTURA DE DADOS: Entrada: body CriarReservaRequest com hospedeId, quartoId, quartoServicoId, quartoNumero, checkInData, checkOutData, pagamentoModo, valorDiaria, metodoPagamento. Saída: ReservaResponse.	
PÓS CONDIÇÕES: Reserva criada, histórico registrado e eventos de domínio publicados conforme a modalidade de pagamento.	

CDU0017	Listar reservas
DESCRIÇÃO: Usuário consulta todas as reservas cadastradas.	
ATOES: administrador, funcionário	
PRÉ-CONDIÇÕES: Serviço deve estar disponível.	
FLUXO BÁSICO: 1. Usuário solicita a listagem de reservas. 2. Usuário envia GET /api/v1/reservas. 3. Sistema busca todas as reservas no repositório. 4. Sistema retorna a lista de reservas em formato DTO.	

FLUXO DE EXCEÇÕES:	
FE01 - Nenhuma reserva encontrada: retorna lista vazia.	
ESTRUTURA DE DADOS:	
Entrada: não há parâmetros.	
Saída: Lista de ReservaResponse.	
PÓS CONDIÇÕES: Lista de reservas retornada ao usuário.	

CDU0018	Buscar reserva por ID
DESCRIÇÃO: Usuário busca uma reserva específica pelo identificador.	
ATORES: administrador, funcionário	
PRÉ-CONDIÇÕES: Usuário possui o ID da reserva.	
FLUXO BÁSICO: 1. Usuário informa o ID da reserva. 2. Usuário envia GET /api/v1/reservas/{id}. 3. Sistema busca a reserva pelo ID informado. 4. Sistema retorna os dados da reserva.	
FLUXO DE EXCEÇÕES:	
FE01 - Reserva não encontrada: retorna erro de recurso não encontrado.	
ESTRUTURA DE DADOS:	
Entrada: idReserva.	
Saída: ReservaResponse.	
PÓS CONDIÇÕES: Reserva retornada ao usuário.	

CDU0019	Listar reservas por hóspede
DESCRIÇÃO: Usuário consulta o histórico de reservas vinculado a um hóspede.	
ATORES: administrador, funcionário	
PRÉ-CONDIÇÕES: Usuário possui o ID do hóspede.	
FLUXO BÁSICO: 1. Usuário informa o ID do hóspede. 2. Usuário envia GET /api/v1/reservas/hospede/{hospedeId}. 3. Sistema busca as reservas do hóspede ordenadas por data de criação decrescente. 4. Sistema retorna a lista de reservas.	
FLUXO DE EXCEÇÕES:	
FE01 - Nenhuma reserva encontrada para o hóspede: retorna lista vazia.	
ESTRUTURA DE DADOS:	

Entrada: hospedeId.
Saída: Lista de ReservaResponse.
PÓS CONDIÇÕES: Histórico de reservas do hóspede retornado ao usuário.

CDU0020	Realizar check-in
DESCRIÇÃO: Usuário altera uma reserva elegível para o status ATIVA.	
ATORES: administrador, funcionário	
PRÉ-CONDIÇÕES: Reserva deve existir. Para pagamento antecipado, a reserva deve estar PAGA. Para pagamento no hotel, a reserva deve estar PENDENTE.	
FLUXO BÁSICO: 1. Usuário informa o ID da reserva. 2. Usuário envia POST /api/v1/reservas/{id}/check-in. 3. Sistema valida a transição de status. 4. Sistema altera a reserva para ATIVA, registra data/hora de check-in, salva histórico e publica evento de check-in. 5. Sistema retorna a reserva atualizada.	
FLUXO DE EXCEÇÕES: FE01 - Reserva não encontrada: retorna erro de recurso não encontrado. FE02 - Reserva com pagamento antecipado ainda não confirmado: retorna erro de regra de negócio. FE03 - Reserva encerrada ou em status incompatível: retorna erro de regra de negócio.	
ESTRUTURA DE DADOS: Entrada: idReserva. Saída: ReservaResponse.	
PÓS CONDIÇÕES: Reserva fica com status ATIVA e check-in registrado.	

CDU0021	Realizar check-out
DESCRIÇÃO: Usuário encerra uma reserva ativa.	
ATORES: administrador, funcionário	
PRÉ-CONDIÇÕES: Reserva deve existir e estar com status ATIVA.	
FLUXO BÁSICO: 1. Usuário informa o ID da reserva. 2. Usuário envia POST /api/v1/reservas/{id}/check-out. 3. Sistema valida a transição de status.	

4. Sistema altera a reserva para ENCERRADA, registra data/hora de check-out, salva histórico e publica evento de check-out. 5. Quando houver quartoServicoId, sistema tenta registrar o check-out no serviço de quartos. 6. Sistema retorna a reserva atualizada.
FLUXO DE EXCEÇÕES: FE01 - Reserva não encontrada: retorna erro de recurso não encontrado. FE02 - Reserva ainda não iniciada ou em status diferente de ATIVA: retorna erro de regra de negócio. FE03 - Falha ao comunicar check-out ao serviço de quartos: erro é registrado em log, sem impedir o retorno da reserva.
ESTRUTURA DE DADOS: Entrada: idReserva. Saída: ReservaResponse.
PÓS CONDIÇÕES: Reserva fica com status ENCERRADA e check-out registrado.

CDU0022	Confirmar pagamento antecipado
DESCRIÇÃO: Usuário confirma manualmente o pagamento de uma reserva com modalidade PAGO_ANTECIPADO.	
ATORES: administrador, funcionário	
PRÉ-CONDIÇÕES: Reserva deve existir, ter modalidade PAGO_ANTECIPADO e estar pendente de pagamento.	
FLUXO BÁSICO: 1. Usuário informa o ID da reserva. 2. Usuário envia POST /api/v1/reservas/{id}/confirma-pagamento. 3. Sistema valida se a reserva permite confirmação de pagamento antecipado. 4. Sistema altera reservaStatus para PAGA e pagamentoStatus para PAGO. 5. Sistema registra histórico, publica evento de pagamento confirmado e retorna a reserva atualizada.	
FLUXO DE EXCEÇÕES: FE01 - Reserva não encontrada: retorna erro de recurso não encontrado. FE02 - Reserva não é de pagamento antecipado: retorna erro de regra de negócio. FE03 - Reserva não está pendente de pagamento: retorna erro de regra de negócio, exceto quando já estiver paga, caso em que retorna sem nova alteração.	
ESTRUTURA DE DADOS: Entrada: reservaId (path variable). Saída: ReservaResponse.	

PÓS CONDIÇÕES: Reserva antecipada fica com pagamento confirmado.

CDU0023	Registrar comentário de encerramento
DESCRIÇÃO: Usuário registra um comentário associado a uma reserva encerrada.	
ATORES: administrador, funcionário	
PRÉ-CONDIÇÕES: Reserva deve existir e estar com status ENCERRADA.	
FLUXO BÁSICO: 1. Usuário informa o ID da reserva e o comentário. 2. Usuário envia POST /api/v1/reservas/{id}/comentarios. 3. Sistema valida se a reserva aceita comentário de encerramento. 4. Sistema salva o comentário vinculado à reserva. 5. Sistema retorna o comentário criado.	
FLUXO DE EXCEÇÕES: FE01 - Reserva não encontrada: retorna erro de recurso não encontrado. FE02 - Reserva não encerrada: retorna erro de regra de negócio. FE03 - Comentário vazio ou com mais de 1000 caracteres: retorna erro de validação.	
ESTRUTURA DE DADOS: Entrada: id (path variable) e body RegistrarComentarioReservaRequest com comentario. Saída: ReservaComentarioResponse.	
PÓS CONDIÇÕES: Comentário de encerramento registrado para a reserva.	

CDU0024	Confirmar pagamento por evento externo
DESCRIÇÃO: Sistema confirma o pagamento de uma reserva antecipada a partir de evento recebido por mensageria.	
ATORES: sistema, serviço de pagamento	
PRÉ-CONDIÇÕES: Evento deve conter reservaId. Reserva deve existir e possuir modalidade PAGO_ANTECIPADO.	
FLUXO BÁSICO: 1. Serviço de pagamento publica evento de pagamento confirmado ou processado com status APROVADO. 2. Sistema consome o evento pela fila configurada no RabbitMQ. 3. Sistema localiza a reserva e confirma o pagamento, quando aplicável. 4. Sistema atualiza a reserva para PAGA e pagamento para PAGO.	
FLUXO DE EXCEÇÕES:	

FE01 - Reserva inexistente ou erro no processamento: exceção é capturada e registrada em log pelo consumer. FE02 - Reserva não é de pagamento antecipado: sistema ignora o evento. FE03 - Reserva já está PAGA, ATIVA, ENCERRADA ou CANCELADA: sistema ignora o evento.
ESTRUTURA DE DADOS: Entrada: PagamentoReservaConfirmadoExternoPayload ou PagamentoProcessadoPayload com reservaId e status APROVADO. Saída: não há resposta HTTP; processamento assíncrono.
PÓS CONDIÇÕES: Pagamento da reserva antecipada é confirmado, quando elegível.

CDU0025	Cancelar reserva por pagamento expirado
DESCRIÇÃO: Sistema cancela uma reserva antecipada quando recebe evento informando expiração do pagamento.	
ATORES: sistema, serviço de pagamento	
PRÉ-CONDIÇÕES: Evento deve conter reservaId e status EXPIRADO. Reserva deve existir e possuir modalidade PAGO_ANTECIPADO.	
FLUXO BÁSICO: - Serviço de pagamento publica evento de pagamento processado com status EXPIRADO. - Sistema consome o evento pela fila configurada no RabbitMQ. - Sistema localiza a reserva e valida se ela pode ser cancelada. - Sistema altera reservaStatus para CANCELADA e pagamentoStatus para EXPIRADO. - Sistema registra histórico da alteração.	
FLUXO DE EXCEÇÕES: FE01 - Reserva inexistente ou erro no processamento: exceção é capturada e registrada em log pelo consumer. FE02 - Reserva não é de pagamento antecipado: sistema ignora o evento. FE03 - Reserva já está PAGA, ATIVA ou ENCERRADA: sistema ignora o evento.	
ESTRUTURA DE DADOS: Entrada: PagamentoProcessadoPayload com reservaId, status e dataAtualizacao. Saída: não há resposta HTTP; processamento assíncrono.	
PÓS CONDIÇÕES: Reserva elegível fica cancelada por expiração do pagamento.	

CDU0026	Cadastrar hóspede
----------------	-------------------

DESCRIÇÃO: Usuário cadastra um novo hóspede no sistema.
ATOES: administrador, funcionário
PRÉ-CONDIÇÕES: CPF deve estar em formato válido e não pode existir outro hóspede com o mesmo CPF.
FLUXO BÁSICO: 1. Usuário informa dados do hóspede. 2. Usuário envia POST /api/v1/hospedes. 3. Sistema valida os campos obrigatórios, normaliza o CPF e verifica duplicidade. 4. Sistema salva o hóspede como ativo. 5. Sistema retorna o hóspede criado.
FLUXO DE EXCEÇÕES: FE01 - CPF inválido: retorna erro de regra de negócio. FE02 - CPF duplicado: retorna erro de conflito. FE03 - Nome, CPF, e-mail ou telefone inválidos: retorna erro de validação.
ESTRUTURA DE DADOS: Entrada: body CriarHospedeRequest com nomeCompleto, cpf, email, telefone. Saída: HospedeResponse.
PÓS CONDIÇÕES: Hóspede cadastrado com status ativo.

CDU0027	Listar hóspedes
DESCRIÇÃO: Usuário consulta todos os hóspedes cadastrados.	
ATOES: administrador, funcionário	
PRÉ-CONDIÇÕES: Serviço deve estar disponível.	
FLUXO BÁSICO: 1. Usuário solicita a listagem de hóspedes. 2. Usuário envia GET /api/v1/hospedes. 3. Sistema busca todos os hóspedes no repositório. 4. Sistema retorna a lista de hóspedes.	
FLUXO DE EXCEÇÕES: FE01 - Nenhum hóspede cadastrado: retorna lista vazia.	
ESTRUTURA DE DADOS: Entrada: não há parâmetros. Saída: Lista de HospedeResponse.	
PÓS CONDIÇÕES: Lista de hóspedes retornada ao usuário.	

CDU0028	Buscar hóspede por ID
DESCRIÇÃO: Usuário busca um hóspede específico pelo identificador.	
ATOES: administrador, funcionário	
PRÉ-CONDIÇÕES: Usuário possui o ID do hóspede.	
FLUXO BÁSICO: 1. Usuário informa o ID do hóspede. 2. Usuário envia GET /api/v1/hospedes/{id}. 3. Sistema busca o hóspede pelo ID informado. 4. Sistema retorna os dados do hóspede.	
FLUXO DE EXCEÇÕES: FE01 - Hóspede não encontrado: retorna erro de recurso não encontrado.	
ESTRUTURA DE DADOS: Entrada: id (path variable). Saída: HospedeResponse.	
PÓS CONDIÇÕES: Hóspede retornado ao usuário.	

CDU0029	Buscar hóspede por CPF
DESCRIÇÃO: Usuário busca um hóspede específico pelo CPF.	
ATOES: administrador, funcionário	
PRÉ-CONDIÇÕES: Usuário possui um CPF válido.	
FLUXO BÁSICO: 1. Usuário informa o CPF do hóspede. 2. Usuário envia GET /api/v1/hospedes/cpf/{cpf}. 3. Sistema valida e normaliza o CPF. 4. Sistema busca o hóspede pelo CPF normalizado. 5. Sistema retorna os dados do hóspede.	
FLUXO DE EXCEÇÕES: FE01 - CPF inválido: retorna erro de regra de negócio. FE02 - Hóspede não encontrado: retorna erro de recurso não encontrado.	
ESTRUTURA DE DADOS: Entrada: cpf (path variable). Saída: HospedeResponse.	
PÓS CONDIÇÕES: Hóspede correspondente ao CPF retornado ao usuário.	

CDU0030	Atualizar hóspede
----------------	-------------------

DESCRIÇÃO: Usuário atualiza os dados cadastrais e o status ativo de um hóspede.
ATORES: administrador, funcionário
PRÉ-CONDIÇÕES: Hóspede deve existir. CPF informado deve ser válido e não pode pertencer a outro hóspede.
FLUXO BÁSICO: 1. Usuário informa o ID do hóspede e os novos dados. 2. Usuário envia PUT /api/v1/hospedes/{id}. 3. Sistema valida CPF, campos obrigatórios e existência do hóspede. 4. Sistema verifica se o CPF não está em uso por outro hóspede. 5. Sistema atualiza nome, CPF, e-mail, telefone e, quando informado, o campo ativo. 6. Sistema retorna o hóspede atualizado.
FLUXO DE EXCEÇÕES: FE01 - Hóspede não encontrado: retorna erro de recurso não encontrado. FE02 - CPF inválido: retorna erro de regra de negócio. FE03 - CPF pertence a outro hóspede: retorna erro de conflito. FE04 - Campos obrigatórios inválidos: retorna erro de validação.
ESTRUTURA DE DADOS: Entrada: id (path variable) e body AtualizarHospedeRequest com nomeCompleto, cpf, email, telefone, ativo. Saída: HospedeResponse.
PÓS CONDIÇÕES: Dados do hóspede atualizados.

CDU0031	Excluir hóspede
DESCRIÇÃO: Usuário exclui um hóspede sem reservas vinculadas.	
ATORES: administrador, funcionário	
PRÉ-CONDIÇÕES: Hóspede deve existir e não pode possuir reservas vinculadas.	
FLUXO BÁSICO: 1. Usuário informa o ID do hóspede. 2. Usuário informa o ID do hóspede. 3. Usuário envia DELETE /api/v1/hospedes/{id}. 4. Sistema busca o hóspede e verifica se existem reservas vinculadas. 5. Sistema exclui o hóspede. 6. Sistema retorna resposta sem conteúdo.	
FLUXO DE EXCEÇÕES:	

FE01 - Hóspede não encontrado: retorna erro de recurso não encontrado. FE02 - Hóspede possui reservas vinculadas: retorna erro de conflito.
ESTRUTURA DE DADOS: Entrada: id (path variable). Saída: resposta HTTP sem corpo.
PÓS CONDIÇÕES: Hóspede removido do sistema.

MATRIZ DE RASTREABILIDADE

CASOS DE USO

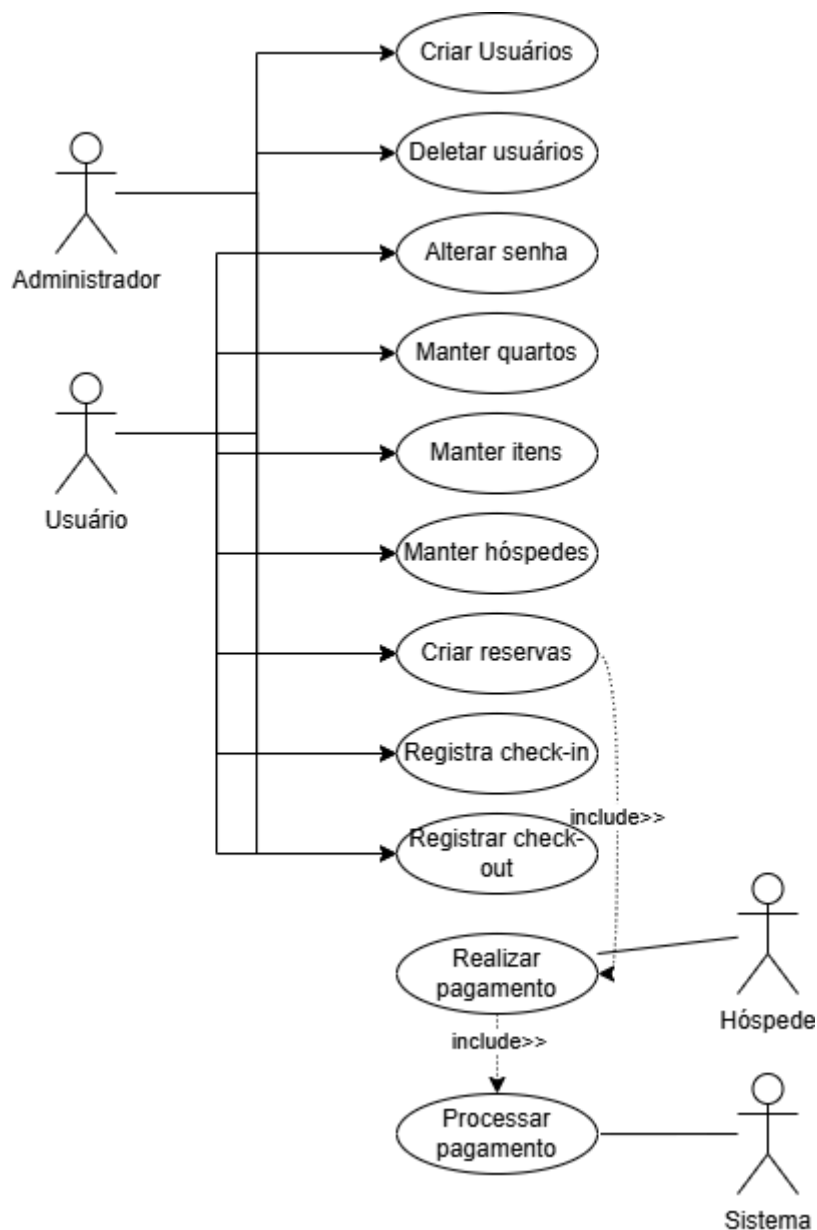
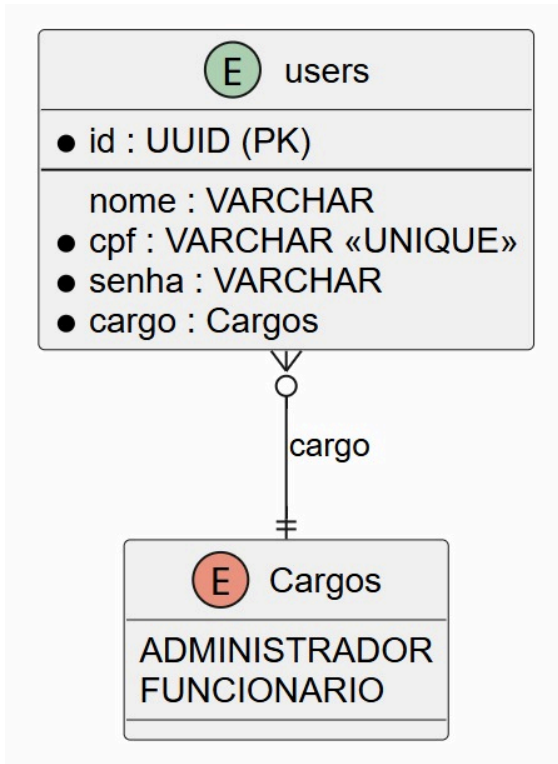
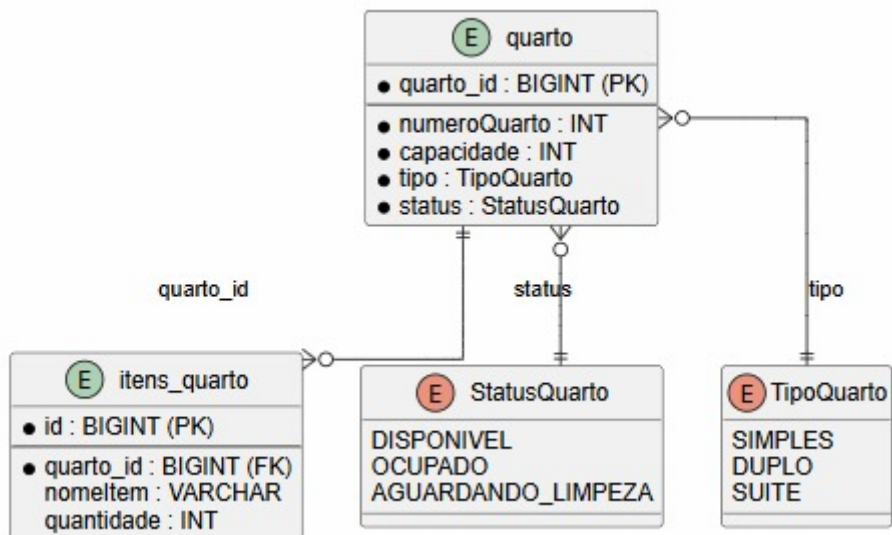


DIAGRAMA ENTIDADE-RELACIONAMENTO

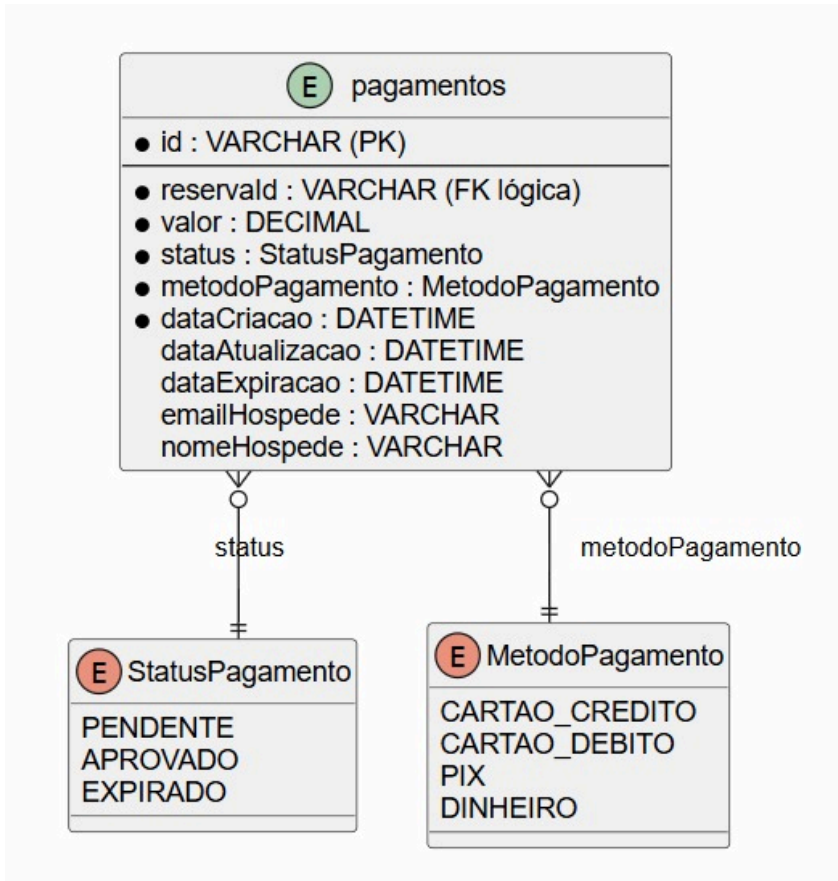
Usuários:



Quarto:



Pagamento:



Sistema:

